LABORATORIO AA. 2025/2026

Computazione Quantistica INF0427

Studio sperimentale dell'algoritmo di Grover

Pierluigi Boscaglia

Matricola: 983553

1 Introduzione

In questo progetto studio l'algoritmo di Grover (paper originale), che permette di cercare una soluzione in uno spazio non strutturato mediante l'amplificazione delle ampiezze. L'obiettivo degli esperimenti è capire come cambia la probabilità di successo al variare del numero di iterazioni e quanto il comportamento ideale si discosta da quello osservato in presenza di rumore.

Il percorso parte dalla costruzione visuale dei circuiti con Quantum Quantum Programming Studio e IBM Quantum Composer e prosegue con l'implementazione in Qiskit, le simulazioni e le prove su hardware IBM Quantum Platform.

2 Primi esperimenti con strumenti visuali

Per iniziare ho costruito due piccoli circuiti attraverso strumenti di composizione visuale: il primo con Quantum Programming Studio, il secondo con IBM Quantum Composer. Ho inserito le porte direttamente sulle linee dei qubit, seguendo da sinistra a destra le operazioni di preparazione, oracolo, diffusore e misura. Questo approccio mi ha permesso di ragionare sulle singole operazioni prima di passare alla costruzione dei circuiti in Python.

Entrambi gli esempi utilizzano due qubit di ricerca, quindi quattro stati possibili. Le porte Hadamard iniziali preparano la sovrapposizione uniforme:

$$|s\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

Inizialmente, ciascuno dei quattro stati ha probabilità 1/4 di essere misurato. L'oracolo inverte il segno dell'ampiezza dello stato cercato, senza modificarne la probabilità; il diffusore sfrutta questa differenza di fase per amplificarne l'ampiezza. In entrambi i circuiti ho applicato una sola iterazione di Grover, composta da oracolo e diffusore. Nel seguito rappresento gli stati nell'ordine $|q_1 q_0\rangle$: il qubit q_0, disegnato in alto nel circuito, corrisponde al bit più a destra nella stringa. Mantengo questa convenzione in tutta la relazione per rendere univoca la lettura degli stati.

2.1 Quantum Programming Studio: circuito senza ancilla

Nel primo circuito ho utilizzato soltanto i due qubit di ricerca. L'oracolo è costruito con porte X, Hadamard e una CNOT. Una Hadamard sul target prima e dopo la CNOT permette di ottenere una porta controlled-Z:

$$H_{q_1}\,\mathrm{CNOT}_{q_0\to q_1}\,H_{q_1} = \mathrm{CZ},$$

dove q_0 è il controllo della CNOT e q_1 il suo target. La CZ lascia invariati gli stati della base, tranne $|11\rangle$, di cui inverte il segno dell'ampiezza. Per marcare invece $|10\rangle$, applico una porta X su q_0 prima e dopo questo blocco. La prima X trasforma temporaneamente $|10\rangle$ in $|11\rangle$; la CZ introduce il segno meno e la seconda X ripristina lo stato iniziale:

$$|10\rangle \xrightarrow{X_{q_0}} |11\rangle \xrightarrow{\mathrm{CZ}} -|11\rangle \xrightarrow{X_{q_0}} -|10\rangle.$$

Gli altri stati vengono ripristinati senza cambiamenti di segno, perché la prima X non li porta in $|11\rangle$.

> L'effetto complessivo dell'oracolo è quindi **invertire soltanto il segno dell'ampiezza** della soluzione $|10\rangle$, **senza modificarne i bit o la probabilità di misura**.

Dopo l'oracolo, il diffusore applica H e X a entrambi i qubit, poi una seconda CZ, ancora realizzata con H–CNOT–H sul target, e infine annulla le X e applica le Hadamard finali. Il qubit target delle CNOT fa parte del registro di ricerca: non viene aggiunto un qubit ausiliario.

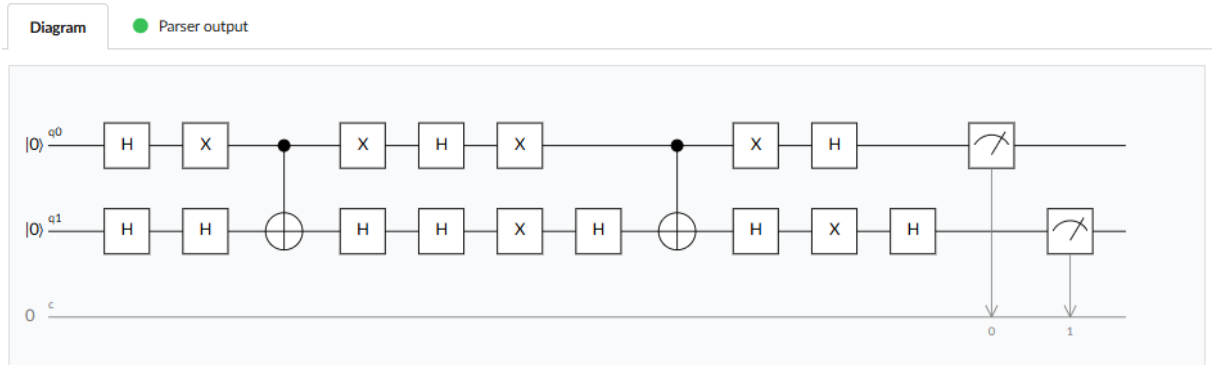


Figura 1: Circuito realizzato in Quantum Programming Studio con due qubit di ricerca, senza ancilla.

2.2 Quantum Composer: circuito con ancilla

Nel secondo circuito ho mantenuto due qubit di ricerca e aggiunto un terzo qubit, indicato con `anc[0]` nello screenshot. Questo qubit non rappresenta un ulteriore bit della soluzione: lo spazio di ricerca rimane di quattro stati. Le porte X e H preparano l'ancilla nello stato

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

L'oracolo è una Toffoli con i due qubit di ricerca come controlli e l'ancilla come target. Poiché $X|-\rangle = -|-\rangle$, quando entrambi i controlli valgono uno si introduce un segno meno nell'ampiezza corrispondente. Nel *phase kickback* l'ancilla permette di modificare la fase dello stato $|11\rangle$, pur rimanendo nello stato $|-\rangle$.

Per costruire il diffusore applico H e X sui qubit di ricerca, riutilizzo la Toffoli con la stessa ancilla, quindi applico nuovamente X e H. Alla fine misuro soltanto i due qubit di ricerca. In Figura 2, l'ancilla è la linea `anc[0]`.

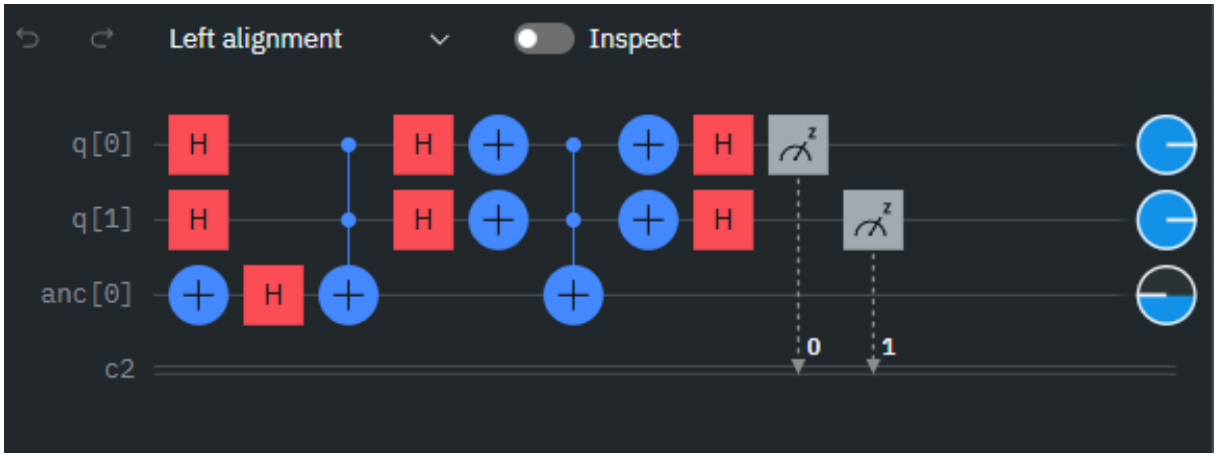


Figura 2: Circuito realizzato in IBM Quantum Composer: due qubit di ricerca e un'ancilla per il phase kickback.

2.3 Confronto e funzionamento dei due circuiti

Le due costruzioni realizzano la stessa idea di Grover con risorse diverse. Il primo circuito modifica direttamente le fasi del registro attraverso una CZ decomposta; il secondo ottiene la modifica di fase usando un'ancilla. Questa differenza dipende dalla costruzione scelta, non da un obbligo imposto dai due strumenti grafici. Inoltre i target sono diversi: $|10\rangle$ nel primo circuito e $|11\rangle$ nel secondo.

Dopo l'oracolo l'ampiezza della soluzione vale $-1/2$ e le altre valgono $1/2$. La loro media è $1/4$: il diffusore standard trasforma ogni ampiezza a in $2\bar{a} - a$, portando quella della soluzione a 1 e le altre a 0. Le sequenze mostrate realizzano il diffusore a meno di un segno globale, che non modifica le probabilità di misura.

Ho verificato le sequenze ricostruite con una simulazione ideale Statevector: il primo circuito restituisce $q_0 = 0$, $q_1 = 1$, mentre il secondo restituisce $q_0 = q_1 = 1$, entrambi con probabilità numericamente pari a 1. Si tratta di una verifica dei circuiti delle immagini, non di conteggi sperimentali raccolti dai due strumenti o di risultati hardware.

Questi primi esempi mostrano quindi che l'ancilla non è indispensabile per costruire Grover: è un modo alternativo di realizzare l'oracolo e il diffusore. In entrambi i casi l'oracolo da solo cambia le fasi, mentre l'amplificazione delle probabilità compare dopo il diffusore.